

5. SYSTEM INTEGRATION GUIDE

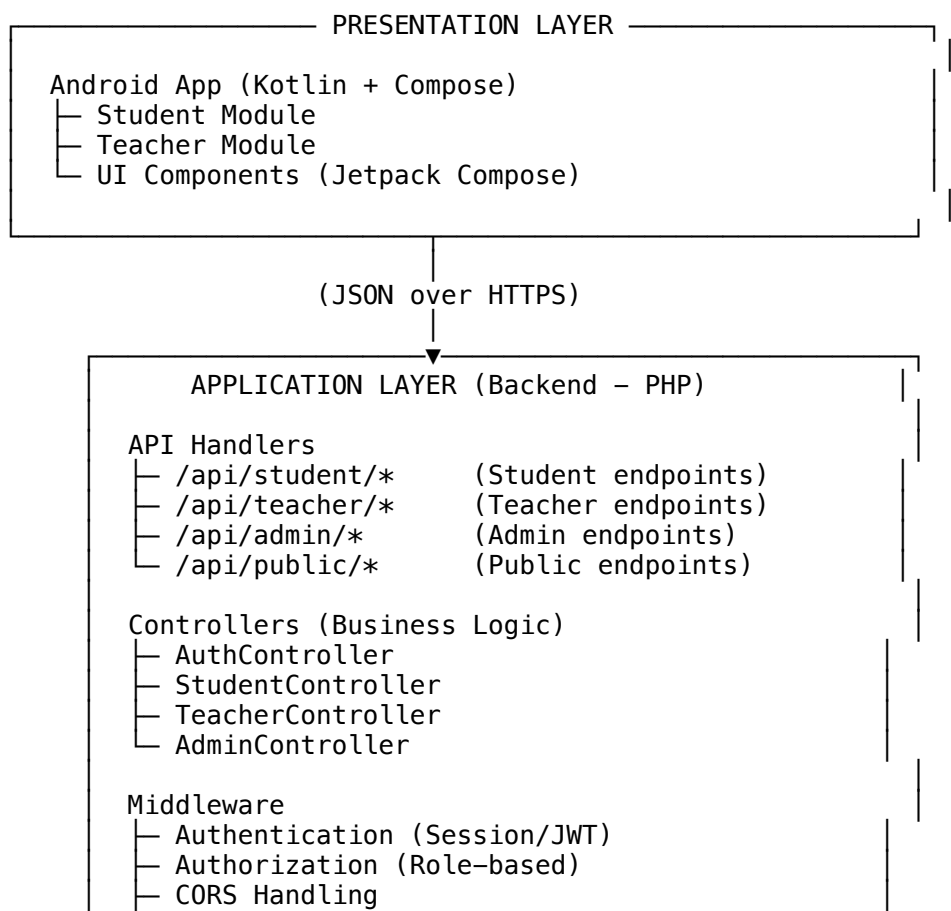
SAMS (Student Attendance Management System) - Complete System Overview

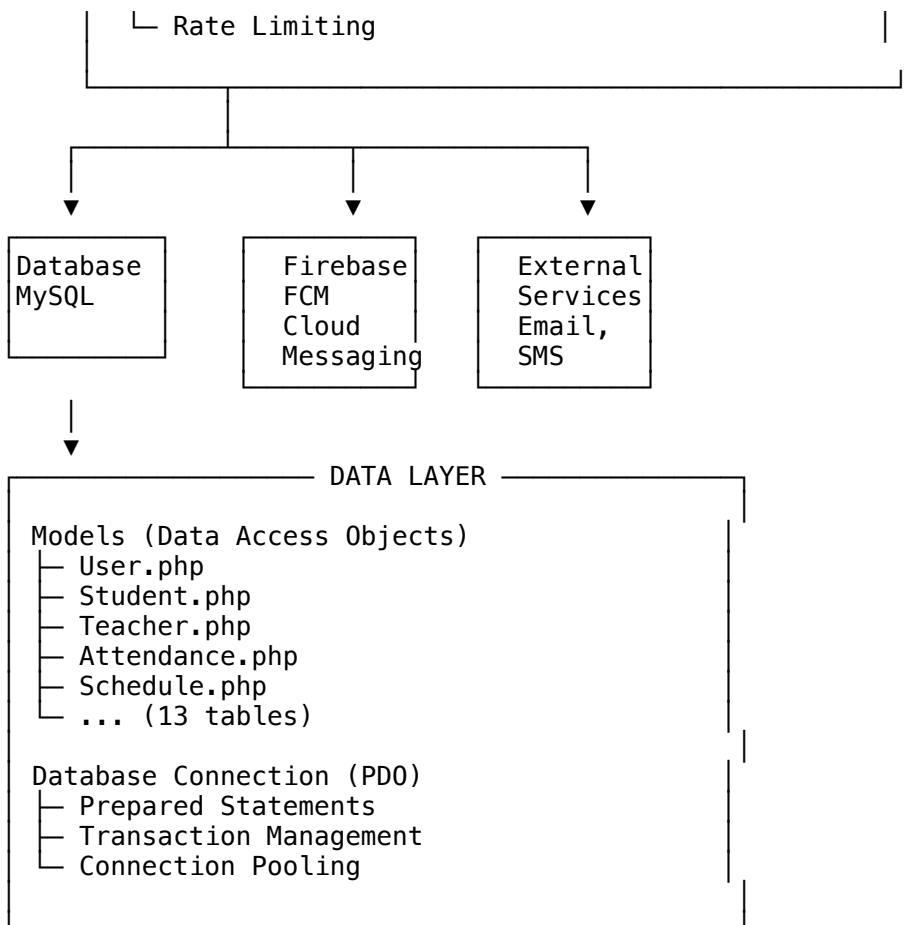
Table of Contents

- 1. [System Architecture](#)
- 2. [Data Flow Diagrams](#)
- 3. [Complete User Journey](#)
- 4. [Component Interactions](#)
- 5. [Deployment Architecture](#)
- 6. [Security Architecture](#)
- 7. [Performance & Scalability](#)
- 8. [Monitoring & Logging](#)
- 9. [Disaster Recovery](#)
- 10. [Best Practices](#)

System Architecture

High-Level Architecture Diagram

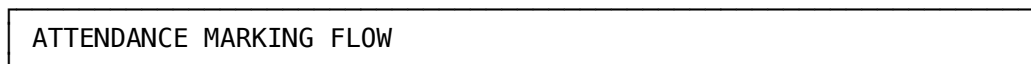




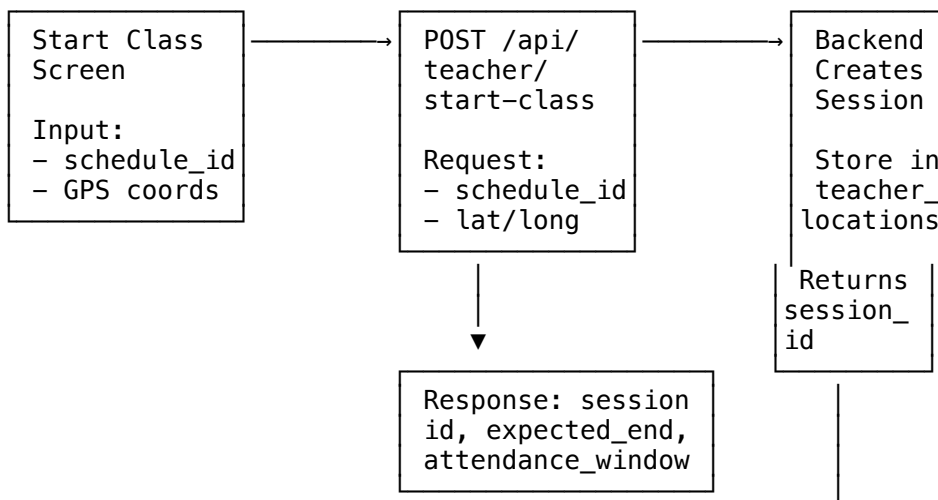
Data Flow Diagrams

1. Attendance Marking Flow

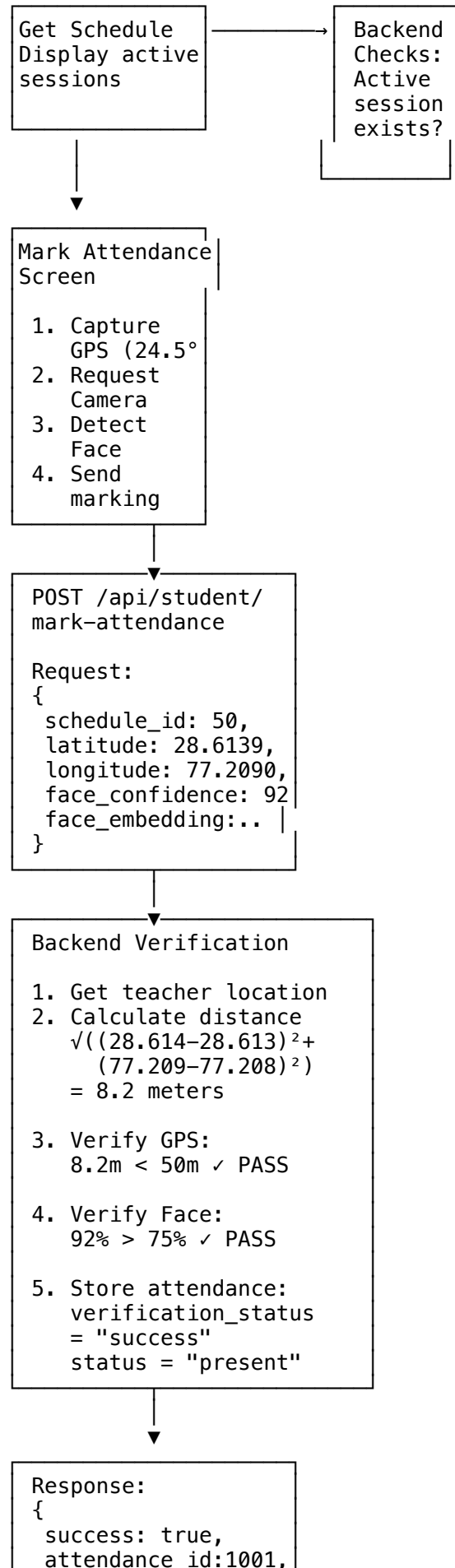
Typical Sequence: Teacher starts class → Students mark attendance → Teacher ends class



TEACHER SIDE:



STUDENT SIDE:

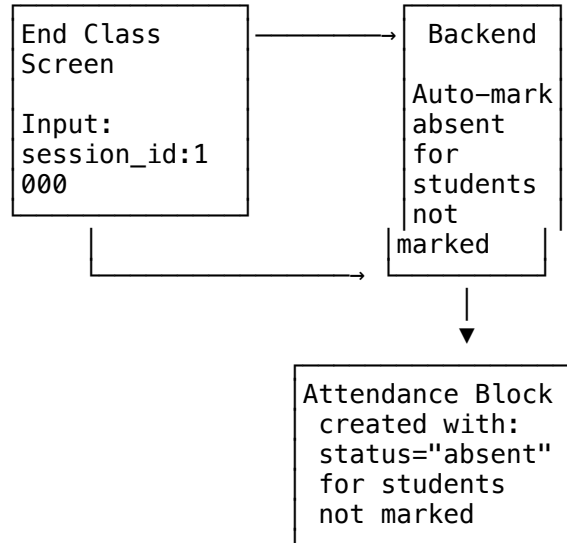


```

status: "present",
verification_status
: "success"
}

```

TEACHER END CLASS:

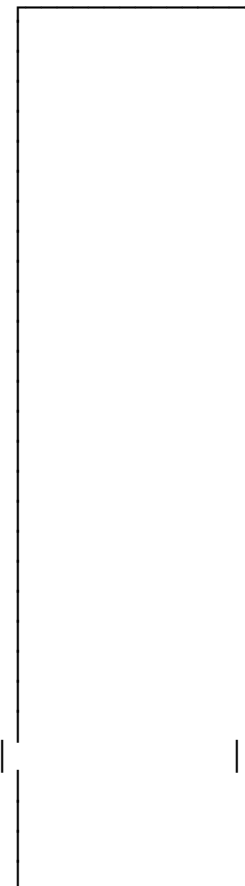
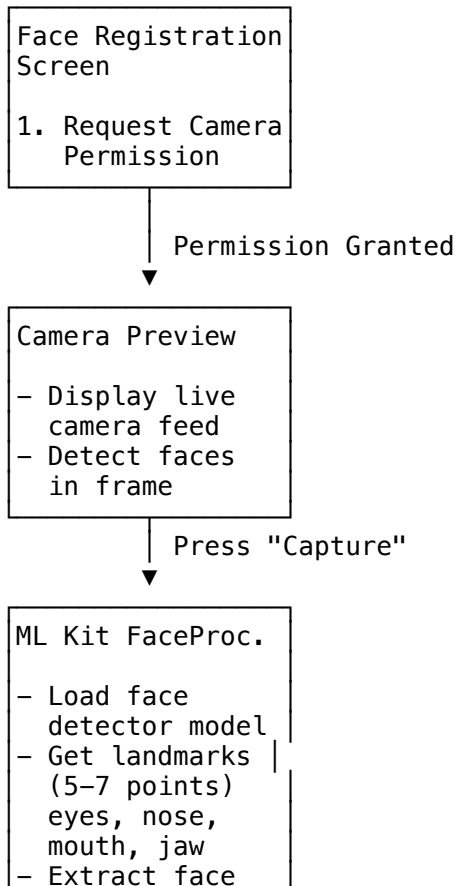


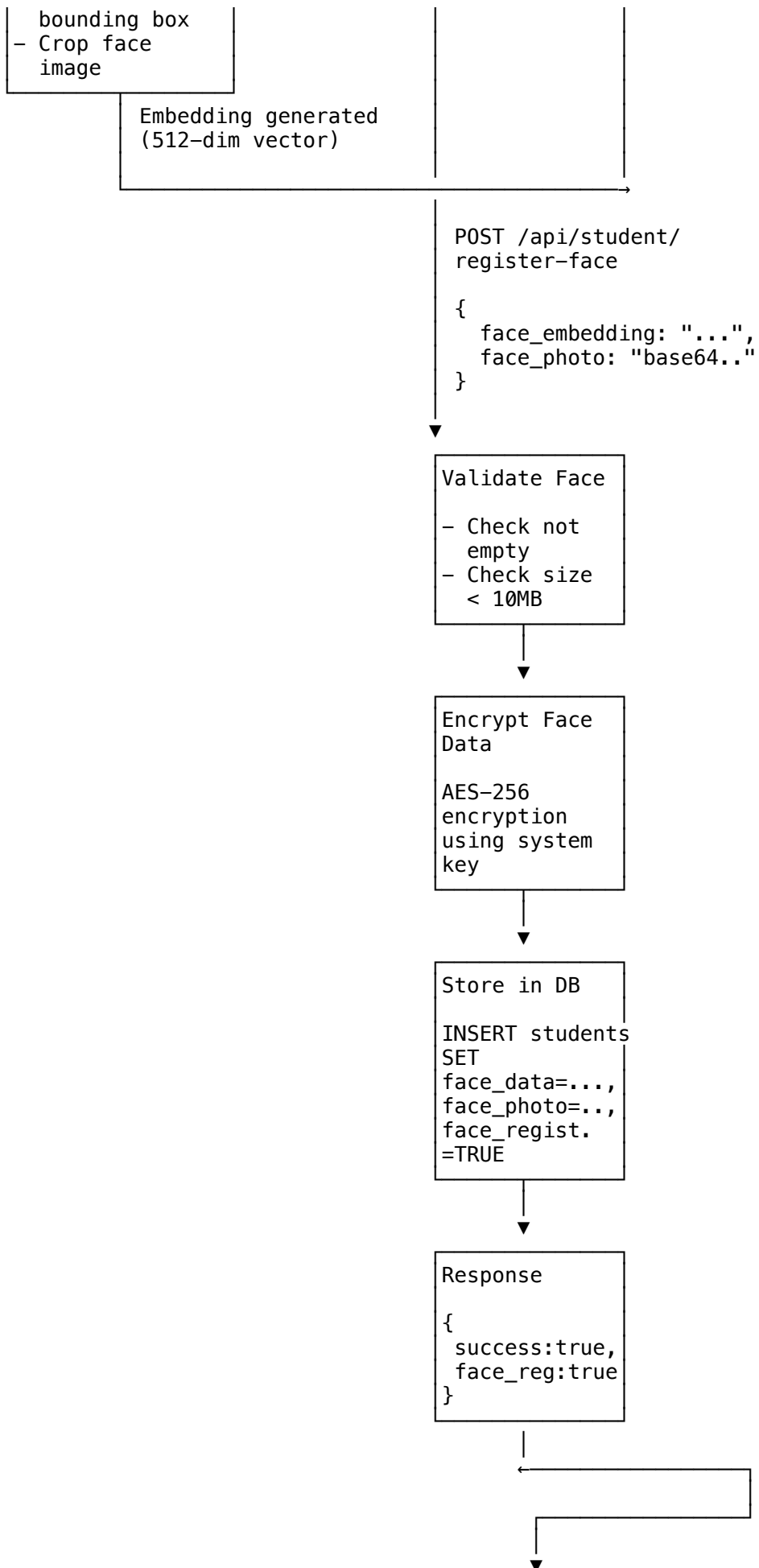
2. Face Registration Flow

FACE REGISTRATION FLOW

STUDENT APP:

BACKEND:





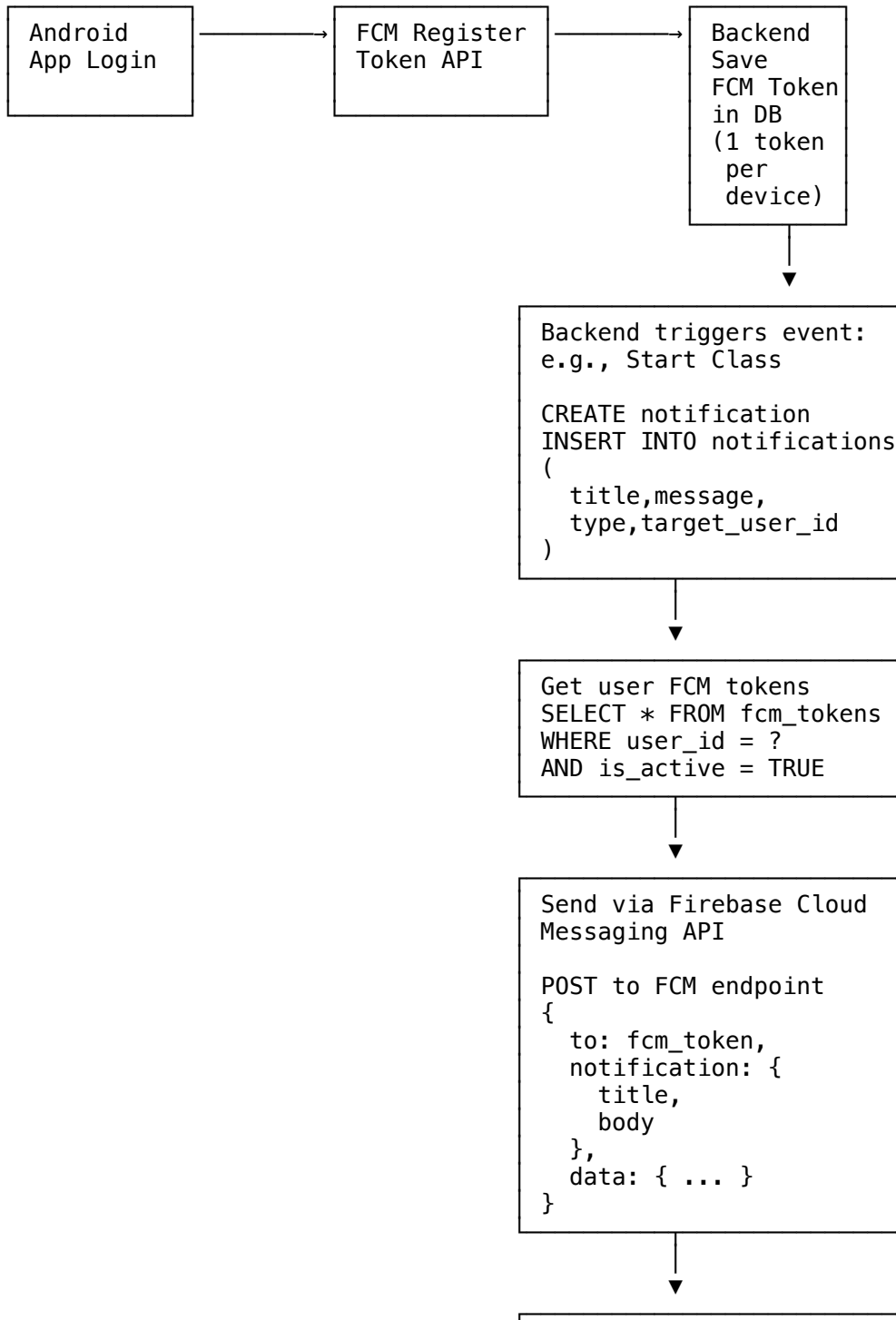
Face Verified
Success!

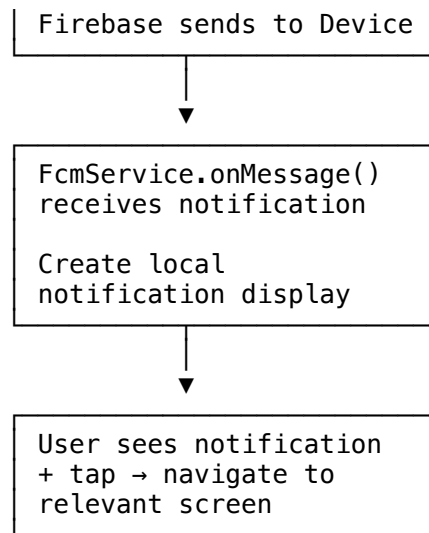
3. Notification Flow (FCM)

NOTIFICATION TRIGGER:

- Low Attendance: Student < 75%
- Schedule Change: Class rescheduled
- Attendance Alert: Class started
- System Alert: Admin announcement

NOTIFICATION FLOW:





Complete User Journey

Student Journey

STUDENT COMPLETE JOURNEY

DAY 1: INSTALLATION & SETUP

- Download SAMS app from Play Store
- Launch app
 - SplashScreen (checks session)
 - No session → LoginScreen
 - Session exists → Dashboard
- LoginScreen:
 - Enter email & password
 - Backend validates credentials
 - Create session (session_id in cookies)
 - Store SessionId locally (SharedPreferences)
 - Navigate to Dashboard
- First Login - Setup:
 - StudentDashboard
 - "Face Registration Required" banner
 - FaceRegistrationScreen:
 - Request camera permission
 - Capture face (multiple angles)
 - ML Kit processes → facial embedding
 - Send to backend
 - Backend: encrypt & store in students.face_data
 - Show "Face registration complete"

DAY 2-N: DAILY USAGE

- Morning class time:
 - Receives FCM notification:
 - "Attendance reminder: Mark attendance for Data Structures"
 - Tap notification → AttendanceMarkingScreen
- AttendanceMarkingScreen:
 - Show active schedules for today
 - User selects schedule
 - Tap "Mark Attendance":
 - Request location permission

- Request camera permission
 - Capture GPS location (lat/long)
 - Send to backend: teacher location check
 - Open camera for face capture
 - ML Kit detects face
 - Extract facial embedding
 - Get face_confidence score (92.5%)
 - Send embedding to backend
 - POST /api/student/mark-attendance
 - Backend verifies GPS (8.2m < 50m ✓)
 - Backend verifies face (92.5% > 75% ✓)
 - Both pass → status = "present"
 - Store attendance record
 - Return success response
- Show: "✓ Attendance Marked Successfully"
 - Navigation back to dashboard

- Mid-day - Dashboard:
 - StudentDashboard shows:
 - Overall attendance: 42/45 = 93.33%
 - Subject-wise breakdown:
 - Data Structures: 14/15 = 93%
 - Algorithms: 14/15 = 93%
 - Web Dev: 9/15 = 60% ⚠
 - "Low Attendance Warning" for Web Dev
 - Recent attendance records
 - Today's schedule
 - ProfileScreen:
 - View profile: Full name, email, roll number
 - Update phone number
 - Update section if applicable
 - ScheduleScreen:
 - View timetable
 - Filter by day of week
 - See teacher names & classrooms
 - AttendanceHistoryScreen:
 - View past attendance records
 - Filter by month/year
 - See verification status
 - "success": Both GPS & Face ✓
 - "gps_failed": GPS out of range x
 - "face_failed": Face confidence low x
 - "both_failed": Both failed x
 - Pagination (20 records per page)

MONTH END:

- Receives notification: "Low attendance in Web Dev!"
- Dashboard shows attendance percentage < 75%
- Parent can check app to see attendance

END OF SESSION:

- Admin marks end of semester
- Attendance frozen
- Student receives notification: "Attendance locked for review"
- Can view final attendance report

LOGOUT:

- Tap menu → Settings → Logout

- └ POST /api/logout
- └ Backend destroys session
- └ Clear local session data
- └ Navigate to LoginScreen

Teacher Journey

TEACHER COMPLETE JOURNEY

9:00 AM – MORNING:

- └ App syncs schedule for the day
- └ TeacherDashboard shows:
 - └ Today's classes: 2
 - └ Total students to mark: $45 + 30 = 75$
 - └ First class: Data Structures (9:00–10:00, Room 101)
 - └ Second class: Algorithms (10:15–11:15, Lab-A)

9:00 AM – CLASS 1 START:

- └ StartClassScreen:
 - └ Show upcoming schedule:
 - "Data Structures – CSE Sem-2, Section A"
 - "8.45 minutes until start"
 - └ Tap "Start Class":
 - └ Request location permission
 - └ Capture current GPS (28.6139, 77.2090)
 - └ Verify classroom location:
 - "Building-A, Room 101 ✓"
 - └ POST /api/teacher/start-class
 - └ Backend creates session in teacher_locations
 - └ session_id = 1001
 - └ Mark is_active = TRUE
 - └ Return expected end time
 - └ Show: "Class Started! ✓"
 - └ "Attendance window: 30 minutes"
- └ ClassAttendanceScreen (in progress):
 - └ Show list of 45 students:
 - └ John Doe – CSE-001-2024
 - Created at 09:02 – ✓ Present (face_confidence: 92%)
 - └ Raj Kumar – CSE-002-2024
 - Not yet marked (grayed out)
 - └ Priya Singh – CSE-003-2024
 - Created at 09:15 – ✓ Present (face_confidence: 88%)
 - └ ... (more students)
 - └ Statistics in real-time:
 - └ Total: 45
 - └ Present: 2
 - └ Absent: 43 (not marked yet)
 - └ Percentage: 4.4%
 - └ Menu options:
 - └ Manual Attendance (for absent slips):
 - └ Select student
 - └ Mark as "present" or "late"
 - └ Save
 - └ Refresh list

10:00 AM - CLASS END:

- └─ Receives notification: "Class time is over"
- └─ EndClassScreen:
 - └─ Show summary:
 - └─ Total students: 45
 - └─ Present: 42
 - └─ Absent: 3
 - └─ Late: 0
 - └─ Not marked: 0
 - └─ Attendance: 93.3%
 - └─ Backend auto-marks absent:
 - └─ "3 students not marked → auto-absent"
 - └─ Tap "Confirm End Class":
 - └─ POST /api/teacher/end-class
 - └─ session_id = 1001
 - └─ Update teacher_locations: is_active = FALSE
 - └─ Auto-mark absentfor students not marked:
 - └─ UPDATE attendance SET status = 'absent'
 - └─ WHERE schedule_id = 50 AND attendance_date = '2024-01-15'
 - └─ AND student_id NOT IN (marked students)
 - └─ Send notification: "Attendance submitted"
 - └─ Show: "✓ Class Ended Successfully"
 - └─ Receives FCM notif: "Attendance for class saved"

10:15 AM - CLASS 2:

- └─ Repeat start/end flow
- └─ Students mark attendance similarly
- └─ Class ends with summary

END OF DAY:

- └─ TeacherDashboard:
 - └─ Today's classes completed: 2/2 ✓
 - └─ Total children taught: 75
 - └─ Average attendance: 92.5%
 - └─ Reports available (download as PDF)
 - └─ Attendance sheet
 - └─ Student-wise details
 - └─ No pending tasks

WEEK VIEW:

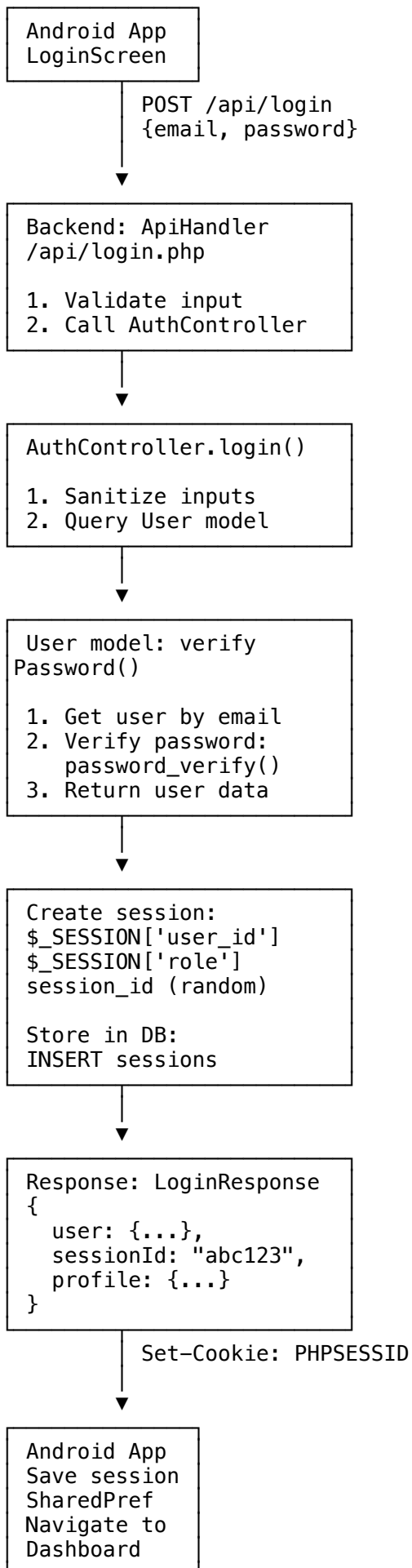
- └─ ScheduleScreen:
 - └─ View all classes this week
 - └─ See completed/pending classes
 - └─ Filter by subject/section
 - └─ Previous attendance records

MONTHLY REPORTS:

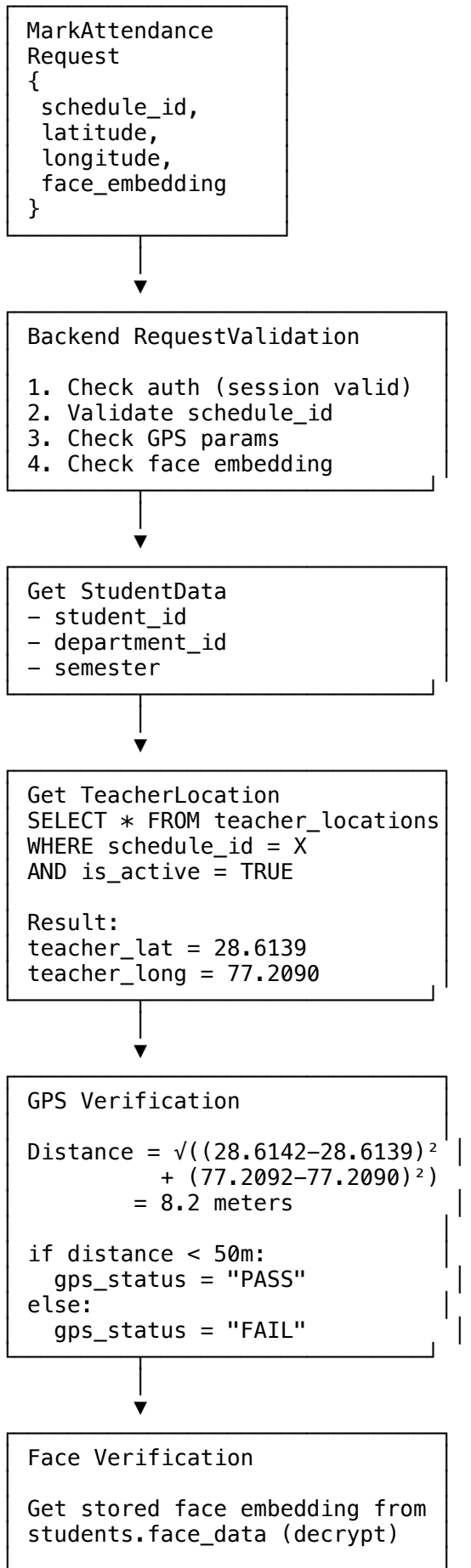
- └─ Admin provides reports:
 - └─ Subject-wise attendance
 - └─ Student-wise details
 - └─ Absentee list
 - └─ Late arrivals list

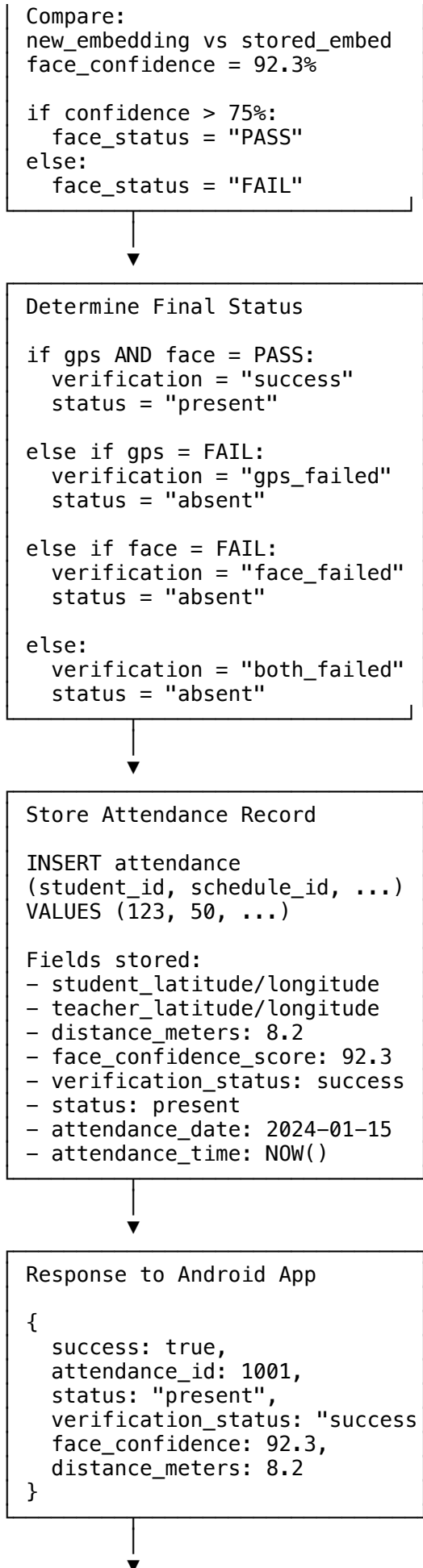
Component Interactions

1. Authentication System



2. Attendance Verification Flow





```
Android App
Show success message
"✓ Attendance Marked"
Navigate back to dashboard
```

Deployment Architecture

Cloud Deployment Options

DEPLOYMENT ARCHITECTURES

OPTION 1: HEROKU (Free/Low Cost)

```
Heroku Dyno (App Container)
├─ PHP built-in server
├─ Composer dependencies
├─ Automatic scaling

JawsDB MySQL
├─ Managed MySQL database

Heroku PgBouncer
├─ Connection pooling

├─ Firebase Cloud Messaging
│   └─ Push notifications
└─ SendGrid
    └─ Email service
```

Azure Deployment

```
AZURE APP SERVICE
├─ B1s or B1ms pricing tier
├─ 1-2 cores, 1-2 GB RAM
├─ 50-100 GB storage
├─ Auto-scaling available

AZURE MYSQL FLEXIBLE SERVER
├─ Standard_B1s tier
├─ 20 GB storage
├─ Burstable performance
├─ Automatic backups

CUSTOM DOMAIN
├─ sams.example.com
├─ SSL/TLS certificate
```

Docker Containerization

```
FROM php:8.1-fpm-alpine
```

```
WORKDIR /app
```

```
# Install dependencies
RUN apk add --no-cache mysql-client
RUN docker-php-ext-install pdo pdo_mysql

# Copy application
COPY . .

# Composer
RUN curl -sS https://getcomposer.org/installer | php
RUN php composer.phar install

EXPOSE 8000

CMD ["php", "-S", "0.0.0.0:8000", "-t", "public"]
```

Security Architecture

Authentication & Authorization

SECURITY LAYERS

LAYER 1: INPUT VALIDATION

- └ Type validation
- └ Length validation
- └ Format validation (email, GPS coords)
- └ SQL injection prevention (prepared statements)

LAYER 2: AUTHENTICATION

- └ Email/password verification
- └ bcrypt password hashing
- └ Session creation with random session_id
- └ Session expiration (1 hour)

LAYER 3: AUTHORIZATION

- └ Role-based access control
 - └ Admin: full access
 - └ Teacher: own classes only
 - └ Student: own data only
- └ Resource ownership checks
 - └ User can only access their own data
- └ Middleware validation on every request

LAYER 4: DATA PROTECTION

- └ Face data encrypted with AES-256
- └ SSL/TLS for API communication
- └ CORS whitelisting
- └ X-CSRF-Token (if using forms)
- └ Rate limiting (prevent brute force)

LAYER 5: SESSION MANAGEMENT

- └ Session stored in database
- └ IP address validation
- └ User agent verification
- └ Concurrent session limits
- └ Secure logout (session destruction)

Data Encryption

SENSITIVE DATA HANDLING:

- └ Passwords
 - └ password_hash() → bcrypt (not reversible)
 - └ Face data
 - └ OpenSSL AES-256-CBC
 - └ Encryption key from config
 - └ IV (initialization vector)
 - └ Encrypted storage in DB
 - └ Session tokens
 - └ bin2hex(random_bytes(32)) → 64-char token
 - └ API communications
 - └ HTTPS/TLS 1.2+
 - └ Database connections
 - └ SSL for remote MySQL servers
-

Performance & Scalability

Database Optimization

INDEXING STRATEGY:

- └ Primary keys (all tables)
- └ Foreign keys (join optimization)
- └ Search columns:
 - └ users.email (UNIQUE)
 - └ students.roll_number (UNIQUE)
 - └ attendance.student_id, attendance_date
 - └ schedules.day_of_week
- └ Filter columns:
 - └ is_active (boolean filters)
 - └ status (enum filters)
- └ Composite indexes:
 - └ (student_id, attendance_date)
 - └ (teacher_id, is_active)
 - └ (department_id, semester)

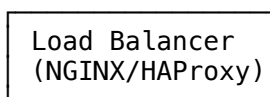
QUERY OPTIMIZATION:

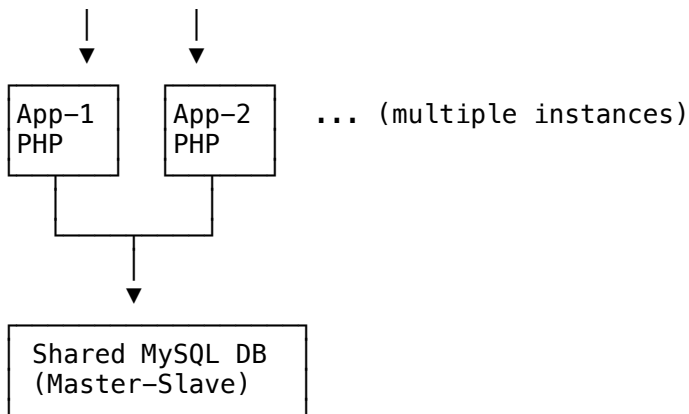
- └ Prepared statements (prevent injection)
- └ Avoid SELECT *
- └ Pagination (limit 50 records per page)
- └ JOIN optimization (use INNER JOIN)
- └ Caching frequently accessed data

EXPECTED PERFORMANCE:

- └ Database queries: < 100ms (indexed)
- └ API response: < 500ms
- └ Login: ~1 second
- └ Dashboard load: ~2 seconds
- └ Bulk attendance mark: ~200ms per record
- └ Scalability: ~5000 concurrent users

Horizontal Scaling

LOAD BALANCER SETUP:



Monitoring & Logging

Application Logging

```
// log_message($level, $message, $context)
log_message('info', 'User logged in', ['user_id' => 123]);
log_message('error', 'DB connection failed', ['error' => $e->getMessage()]);
log_message('debug', 'Face verification complete', ['confidence' => 92.5]);
```

Analytics Events

EVENT TRACKING

User Login:

- timestamp
- email
- role
- device_info

Attendance Event:

- student_id
- schedule_id
- timestamp
- verification_status
- face_confidence
- gps_distance

Face Registration:

- student_id
- timestamp
- success/failure

Class Session:

- teacher_id
- schedule_id
- start_time
- end_time
- attendance_count

Disaster Recovery

Backup Strategy

BACKUP FREQUENCY:

- └ Database
 - └ Daily full backup
 - └ Hourly incremental
 - └ Storage: AWS S3 / Azure Blob
- └ Application code
 - └ Git repository (GitHub/GitLab)
 - └ Weekly zips to cloud storage
- └ Media files
 - └ Daily sync to S3/Blob

RECOVERY PLAN:

- └ RTO (Recovery Time Objective): 1 hour
 - └ RPO (Recovery Point Objective): 1 hour
 - └ Failover: Automated to secondary region
 - └ Testing: Monthly disaster recovery drills
-

Best Practices

Code Quality

STANDARDS:

- └ PSR-12 PHP coding standards
- └ Type hints for all functions
- └ Comprehensive error handling
- └ Logging at important checkpoints
- └ Input validation on all endpoints
- └ Prepared statements for all queries

TESTING:

- └ Unit tests: Controllers & Models
- └ Integration tests: API endpoints
- └ Manual tests: Critical workflows
- └ Load tests: Expected user concurrency

DOCUMENTATION:

- └ API endpoint documentation (OpenAPI/Swagger)
- └ Database schema documentation
- └ Deployment guides
- └ Security guidelines
- └ Troubleshooting guides

Security Best Practices

ONGOING SECURITY:

- └ Regular security audits
 - └ Dependency updates (composer)
 - └ PHP version updates
 - └ SSL certificate renewal
 - └ Password policy enforcement
 - └ Intrusion detection monitoring
 - └ OWASP Top 10 compliance
 - └ Penetration testing (annually)
-

Summary

SAMS System Integration:

1. **Frontend** (Android): Jetpack Compose, Kotlin, ML Kit
2. **Backend** (PHP): RESTful API, MVC architecture
3. **Database** (MySQL): 13 tables, normalized schema
4. **Services** (Firebase): FCM for notifications
5. **Communication** (HTTPS): Secure JSON API

Core Flows: - Authentication → Session creation → API calls - Attendance marking → GPS + Face verification → Storage - Notifications → FCM token → Push delivery - Reports → Data aggregation → Export

Security: Input validation → Authentication → Authorization → Encryption

Scalability: Database indexing → Query optimization → Load balancing → Horizontal scaling

Operations: Monitoring → Logging → Backups → Disaster recovery